SAMS Kernel

The workbench user manual: editing, assembling and running on the TI

Version 1.35.1 · 28 October 2026
For a TI-99/4A with an F18A video card and a SAMS memory card
MIT License, © 2026 Rebecca Sutter

Contents

Contents

1. What it is, and what it is not

2. What you need

3. Starting it

4. The screen and the menu

5. Files, names and disks

6. The editor

7. The assembler

8. Running a program, and getting back

9. Tutorials

10. The programs on the disks

11. Messages and what to do about them

12. Limits at a glance

13. Quick reference

14. The PC route, and where to read next

The workbench is a program for the TI-99/4A itself: an **editor**, an **assembler** and a **launcher** in one application. With it you write a SAMS Kernel program, assemble it and run it, on the TI, with no PC once the workbench is installed. This is the manual for using it. `docs/WORKBENCH.md` is the shorter technical guide to how it is built and what it costs; the programmer's guide teaches the programs you will write with it.

What has been checked, and what has not. Everything in this manual runs in a simulator (a TMS9900, the F18A, SAMS, a keyboard, TI's own console ROM and disk controller ROMs), and every screen shown here is a real screen from that simulator. **The workbench has not run on a real TI-99/4A**, and no time has been measured on one; `docs/HARDWARE-CHECKLIST.md` says how to try it.

Contents

1. What it is, and what it is not
2. What you need
3. Starting it: the cartridge, or the disk
4. The screen and the menu
5. Files, names and disks
6. The editor
7. The assembler
8. Running a program, and getting back
9. Tutorials
10. The programs on the disks
11. Messages and what to do about them
12. Limits at a glance
13. Quick reference
14. The PC route, and where to read next

---

1. What it is, and what it is not

You can do four things with it: **name a file** on a disk, **edit** it, **assemble** it, **run** the result. A file is a plain text source in the assembly language the kernel's examples use (xas99's dialect; part 7). The result runs *inside* the workbench's memory, as a kernel module, and gives control back when it finishes.

It is meant for: learning the kernel by changing the example programs; writing and trying small programs away from a PC; keeping a TI-only workflow going; and checking that a source assembles on the machine it is for.

It cannot: write a **cartridge image** (a FinalGROM's storage is not something the TI can write); build a **disk-startable program** (`skbuild --disk` runs on a PC); assemble a program of more than one module, with its own data pages, music or font (part 12); assemble user macros, `DEF`/`REF` or relocatable output; or produce a listing. A program for more than the workbench can take is built on a PC with `skbuild.py`; the source is the same either way, and the guide's part 0 compares the routes.

2. What you need

a **TI-99/4A**	with an **F18A** video card (a PICO9918 also works) and a **SAMS** memory card of at least 512K; the workbench refuses to start with a message if either is missing
a way to **start it**	a **FinalGROM 99** (or js99er or Classic99 with the F18A and SAMS settings on) holding `workbench8.bin`; or **no cartridge at all**: a disk-started workbench, Editor/Assembler option 5 (part 3)
a **disk** in DSK1	a floppy, a CF7+ or a TIPI: it holds `KERNEL/EQU`, your sources and anything you save

All of these are in the distribution's `images/` directory: `workbench8.bin` (the cartridge), `workbench.dsk` (90K single density, for TI's own controller), `workbench-360k.dsk` (360K double density, for the CorComp, Myarc and HFDC cards, a CF7+ or a TIPI: everything in one place) and `workbench-diskstart-360k.dsk` (the workbench as a disk-started program, with its sources: 360K, so a double-density controller, a CF7+ or a TIPI; TI's own controller needs a single-density build, part 3). A PC is needed once, to put these on your drive or card; how they get there depends on your hardware, and none of the ways has been tried here.

`KERNEL/EQU` must come from the same build as the workbench. It holds the addresses of the kernel's routines and

equates in that build, and a program assembled against another build's addresses will not run. Every disk image in the distribution carries the right one; if you build your own disk, use the `KERNEL/EQU` that `skbuild.py` or `skdisk.py` writes for the workbench you are using.

3. Starting it

From the cartridge

Put the cartridge in (or load `workbench8.bin` in the emulator), switch on, and choose **WORKBENCH** from the console's title screen. Put the disk in DSK1 first.

RUN PROGRAM FILE: the same cartridge as an Editor/Assembler option 5

The workbench cartridge (`workbench8.bin`) lists **two programs** on the console's menu: **WORKBENCH** and **RUN PROGRAM FILE**. The second is Editor/Assembler option 5, so the cartridge replaces the Editor/Assembler for running programs. It shows a prompt with `DSK1.` already in the name line; type the rest of a file name (`DSK1.GAME`, `DSK2.UTIL1`, a TIPI path), `FCTN S` or `FCTN 1` deletes a character, **ENTER** loads and runs, `FCTN =` restarts the console. It loads the file with the disk controller, copies it to its load address, goes on to the next file of the chain while the first word of a file says there is one (the next name is the last character plus one, as the Editor/Assembler does), and then calls the first file's load address with the GPL workspace and interrupts on, as option 5 does; a program that ends with `RT` comes back to the prompt. **It runs two kinds of program with the one mechanism**: an ordinary option 5 program (a memory image: `xas99 -i` makes them, and the Editor/Assembler's SAVE), and **a kernel disk program**, whose bootstrap file is such an image (`skbuild.py --disk`): type `DSK1.HELLO` and the bootstrap reads its banks and starts the kernel. **ENTER with the name line still `DSK1.` runs `DSK1.UTIL1`**, as the Editor/Assembler does with an empty name. Before the program is called it sets the screen up as the module does (colour table black on light green, screen blank, backdrop white on light green). The prompt has a blinking cursor, and `LOADING...` shows while a file loads. **The workbench cartridge also runs module programs** (docs/MODPROG.md): `DSK1.GUEST` for a file made by `skmodprog.py` against the cartridge's `kernel.equ` boots the cartridge's kernel with the program's module as the application; a program built for another build is refused with both build IDs on the screen. `images/modprog-demo.dsk` has an example. It does not load option 3 object code (the `DF80` files of the Editor/Assembler's loader), and a program that calls the Editor/Assembler's utilities (`VMBW`, `KSCAN`, `DSRLNK`, loaded by that module into low RAM) must carry them, as an image from the Editor/Assembler has to. Errors: `I/O ERROR n` with the disk controller's code (7: the file is not there or is not a program file) or `NO SUCH DEVICE`. Tested in the simulator against a two-file image and a kernel disk program; **not tried with any real program or on a real TI** (`docs/HARDWARE-CHECKLIST.md`, steps R1 to R4).

From the disk, with no cartridge

The disk-started workbench is an Editor/Assembler option 5 program. With the Editor/Assembler cartridge: choose **option 5, RUN PROGRAM FILE**, and type

```
DSK1.WBENCH
```

The bootstrap checks the machine, reads the workbench's banks (`WBENCH/B01` to `B05`) from the disk into SAMS, and starts it: about 24K from the disk (its bank files are trimmed, not padded to 8K each), a time that is not known. Everything must be on the one disk or volume (DSK1 to DSK3 are tried): the program, its bank files, `KERNEL/EQU` and your sources. Build it on a PC with

```
python3 tools/skbuild.py examples/workbench --disk --geometry dsdd --disk-files examples/hello/hello.a99 ...
```

(`--geometry sssd` for TI's own single-density controller, which holds the workbench and room for a few small sources). `docs/DISKSTART.md` has the design and what is assumed about a real machine.

What you see first

```
  SAMS Kernel   workbench

  File: DSK1.

  N  type a file name          C  the catalog of the disk
  E  edit the file             A  assemble it         R  run the program
  FCTN =  quit
```

If the machine cannot run it you see a **refusal screen** instead, saying what is missing: no F18A, too few SAMS pages, a

SAMS page that does not hold what is written to it, or (disk start) that the program's files cannot be read from the disk. `docs/DEVELOPING-ON-A-TI.md` has the ladder of what to try.

4. The screen and the menu

The screen is 80 columns by 30 rows of text with colour. The menu is the top of it; below the key list the **result** of the last assembly appears.

Key	What it does
`N`	type a **file name** (part 5): the field starts as `DSK1.` and you add to it
`C`	the **catalog** of the disk named in the file name
`E`	**edit** the file named (a new file if it is not on the disk)
`A`	**assemble** the file named, as it is *on the disk*; the result appears below
`R`	**run** the program just assembled
`FCTN =` (QUIT)	leave the workbench: the console restarts

Keys are the console's own: **FCTN** and **CTRL** are the keys at the bottom left; `FCTN 9` is BACK, `FCTN =` is QUIT, and so on (the table in part 13). The keyboard's **ALPHA LOCK** works as usual: down gives capitals (all the screens in this manual were typed with it down); up gives lower case, and whatever you type is what is saved. The assembler does not care about case, except inside a quoted string.

After an assembly the menu shows either

```
Assembled:  50   bytes at >C000
```

or a list of errors (part 11). Messages that appear on that line when you press a key that cannot be used yet:

Message	Meaning
`nothing assembled to run`	`R` before any successful `A`
`unsaved changes: E, CTRL S, then run`	you edited since the assembly; save (and assemble again) first
`the program must be at >C000 and fit 8K`	the assembled image does not start at `>C000` (a module's address)

5. Files, names and disks

TI file names

A TI file name is up to 10 characters and **has no dots** (the dot separates the device from the name). A source called `hello.a99` on a PC is `HELLO/A99` on the TI: the slash stands in for the dot (`xdm99` and `skdisk.py` make this change when they put a PC file on a disk image). So the name to type is `DSK1.HELLO/A99`.

The file name field (`N`)

Key	
letters, digits, punctuation	typed at the end of the name (lower case becomes capitals)
`ENTER`	done: the name is kept
`FCTN S` (left) or `FCTN 1` (DEL)	delete the last character
`FCTN 9` (BACK)	leave the field

The name may be up to 31 characters, so a path on a TIPI works (`PI.PROJ.HELLO/A99`, `DSK1.` and the like). Only the end of the name can be edited: to change the device, delete back to it.

```
SAMS Kernel   workbench

File: DSK1.MINE/A99#
```

(The `#` is the cursor.)

The catalog (`C`)

`C` lists the files on the disk of the device in the name field (the part up to the last dot): up to the first **20** names. For a disk holding six files:

```
 SAMS Kernel   workbench

 UP DOWN pick a file, ENTER chooses it, BACK returns

   BAD/A99
   BEEP/A99
   HELLO/A99
   KERNEL/EQU
   MODE/A99
   PAGING/A99
```

`UP` and `DOWN` (`FCTN E` and `FCTN X`) move the highlight; **ENTER** puts the chosen file's name in the field (with the device kept); **FCTN 9** returns without choosing. A disk that cannot be read, or has no files, says `no files, or the disk could not be read`. The catalog is read afresh each time; a longer directory shows only its first 20 files, so keep working disks small or put sources on a directory a TIPI can map.

Includes: `COPY`

A source can `COPY "name"` another file. The name is looked for **beside the main file first** (on the same device and directory, dots turned to slashes: `COPY "kernel.equ"` reads `KERNEL/EQU`), then on the bare device (`DSK1.KERNEL/EQU`). Every program starts with `COPY "kernel.equ"`: that is how it learns the kernel's routines and equates.

Disks

Image	Format	Holds
`workbench.dsk` (SKWORKBNCH)	90K, single sided, single density	`KERNEL/EQU`, sample `SRC`, the small programs of part 10
`examples.dsk` (SKEXAMPLES)	90K single density	the two long programs, Reversi and the display modes
`workbench-360k.dsk` (WORKBENCH)	360K, double sided, double density	everything, and the workbench's own sources

The two 90K disks each carry their own `KERNEL/EQU`: put the one you want in drive 1. TI's own disk controller reads single density only.

6. The editor

`E` opens the file named in the field. If the file is not on the disk, the editor starts with an empty one and creates it when you save.

```
 Edit  DSK1.MINE/A99                    line 13   of 13     col 30    INS     *
       COPY "KERNEL.EQU"
       COPY "MACROS.A99"
       AORG >C000
       DATA INIT,TICK,INIT,0
INIT   .ENTER
       LI   R0,2*COLS+2
       LI   R1,MSG
       LI   R2,(RTEXT*16+RBG)*256
       BL   @PUTS
       .LEAVE
       RT
TICK   RT
MSG    STRI 'BUILT ON THE TI'
                                    (the rest of the screen is blank)
 CTRL S save  CTRL A save+assemble  CTRL X menu  CTRL H backspace  FCTN 3 line
```

The top line shows the file, the cursor's line (`line 13 of 13`), its column, the **insert/overwrite** mode (`INS` or `OVR`) and a `*` at the right while there are **unsaved changes**. **The bottom line** is the key help; messages (`saved`, and the ones below) replace it for a moment. The editor holds a file of up to **2,040 lines** of **80 columns**: a longer file is loaded in part (the screen says so) and **saving is refused**, so that the file on the disk is never cut.

The editor does not indent for you: a source line starts with a label at column 1, or with spaces, and the operation comes after the spaces (part 7: this matters).

Keys

Key	What it does
arrows: `FCTN E` up, `FCTN X` down, `FCTN S` left, `FCTN D` right	move the cursor
`FCTN 5` (BEGIN)	to the start of the line
`FCTN 6` (PROC'D)	to the end of the line
`FCTN 7` (AID)	up 20 lines
`FCTN 8` (REDO)	down 20 lines
any character	**inserts** it at the cursor
`FCTN 2` (INS)	switch between inserting and overwriting
`ENTER`	splits the line at the cursor (the rest goes to a new line below)
`CTRL H`	deletes the character **before** the cursor; at the start of a line it joins the line to the one above (if both fit in 80 columns)
`FCTN 1` (DEL)	deletes the character **under** the cursor
`FCTN 3` (ERASE)	deletes the whole line
`CTRL S`	**saves** to the disk (trailing blanks are cut)
`CTRL A`	saves, goes to the menu and **assembles**
`CTRL X`	back to the menu; **press it twice** to leave with unsaved changes

Typing a lower-case letter gives lower case with ALPHA LOCK up and a capital with it down, as everywhere on the console. Everything is saved exactly as typed.

Messages

Message	Meaning
`saved`	`CTRL S` wrote the file
`unsaved changes: CTRL S saves, CTRL X again leaves without saving`	you pressed `CTRL X` with a `*` showing
`the file could not be written`	the disk is full, protected, or not there: nothing was lost from the screen; fix it and save again
`the file is longer than 2,040 lines: only part is loaded and it cannot be saved`	edit a copy, or split the file

```
 Edit  DSK1.MINE/A99                   line 1    of 1      col 4     INS     *
ABC
                                     (blank rows)
 unsaved changes: CTRL S saves, CTRL X again leaves without saving
```

How it holds a file

The lines live in SAMS pages from the kernel's page pool, 51 lines of 80 bytes a page, in the order of a table: inserting or deleting a line moves only the table. That is why a 2,040-line file edits as fast as a short one, and the reason the programmer's guide uses the editor as its worked example (appendix C).

7. The assembler

`A` (or `CTRL A` in the editor) assembles the file **as it is on the disk**, in two passes (the first gives every label its address, the second makes the bytes), reading the file twice from the disk. It reads xas99's dialect, the part the examples use.

A line

```
LABEL  OPERATION  OPERANDS  comment
```

- A **label** starts at **column 1** (an optional colon after it). A line that starts with a space has no label.
- The **operation** comes after at least one space. **A mnemonic at column 1 is taken for a label** (see below).
- A line whose first character is `*` or `;` is a comment; so is anything after the operands.

- Case does not matter (`mov r1,r2` is fine); symbols are **up to 12 characters**, upper and lower case the same.

The mistake that costs the most time: an unindented instruction. `MOV R1,R2` starting in column 1 is read as *the label MOV* followed by *the operation* `R1,R2`, and the assembler says `unknown operation`. Worse, `RT` alone at column 1 is read as a label named RT and **raises no error at all**: the program is shorter than you meant. If the image is smaller than expected, look for an operation at column 1.

Instructions

Every TMS9900 instruction:

arithmetic	`A AB ABS AI C CB CI DEC DECT DIV INC INCT MPY NEG S SB`
logic	`ANDI COC CZC INV ORI SOC SOCB SZC SZCB XOR`
moves	`CLR LI LWPI LIMI MOV MOVB SETO STST STWP SWPB`
shifts	`SLA SRA SRC SRL`
jumps	`JEQ JGT JH JHE JL JLE JLT JMP JNC JNE JNO JOC JOP`
calls	`B BL BLWP RTWP X XOP`
bits and I/O	`LDCR SBO SBZ STCR TB CKOF CKON IDLE LREX RSET`
pseudo	`NOP` (a `JMP $+2`), `RT` (a `B *R11`)

Do not execute `CKOF`, `CKON`, `IDLE`, `LREX` or `RSET` on a TI-99/4A. They assemble, but TI's technical data forbids them on this machine: the CRU clock is only partly decoded, so the peripherals take these instructions for CRU operations. (`PROBE1/A99` contains them only to test the assembler.)

Operand forms: `Rn` (a register), `*Rn` (indirect), `*Rn+` (indirect, then add), `@expr` (symbolic) and `@expr(Rn)` (indexed); immediate values are the second operand of `LI`, `AI`, `ANDI`, `ORI`, `CI`, `LWPI`, `LIMI`. Instructions and `DATA` are placed on **even addresses** (a pad byte is added) and a label on such a line takes the aligned address.

Directives

`AORG expr`	where the program starts: a module is `AORG >C000`. The expression must be defined already
`DATA a,b,...`	words
`BYTE a,b,...`	bytes (-128 to 255)
`TEXT 'abc'`	the characters of the string
`STRI 'abc'`	a length byte, then the characters
`EVEN`	pad to an even address
`BSS n`	reserve n zero bytes
`label EQU expr`	give the label a value: **the value must be defined already** (xas99's rule)
`COPY "name"`	assemble another file here (up to three levels; part 5)
`END`	stop
`LIST UNL TITL IDT PAGE`	accepted and ignored
`.enter .leave .push R .pop R .intsvc .callmod page,entry`	the kernel's macros, **built in**; `COPY "macros.a99"` is accepted and ignored

Not supported: your own macros (`.defm`), `DEF`, `REF`, relocatable output, listings. **The `.mape`, `.mapf` and `.mapd` macros of `macros.a99` (1.20 and 1.22) are not built in**: in a source for the workbench call the routines instead (`LI R0,page` / `BL @MAPE`).

Quote a string with single quotes; a quote inside is doubled (`'it''s'`). `STRI ''` (empty) is not accepted.

Expressions

`+ - * /`, parentheses, decimal numbers, `>hex`, `'c'` and `'cc'` (character codes), symbols, and `$` (the address where the statement starts). **They are evaluated strictly left to right**, as xas99 does: `2+3*4` is **20**, not 14; write `2+(3*4)`.

What it makes

One image of up to **4,096 bytes** in one run from the first `AORG` (gaps between `AORG`s are zero-filled), up to **580 symbols** (the kernel's 341 equates take most of them in an application of the workbench's own size: roughly 240 are yours). The assembler's own work area lives in SAMS pages and does not touch your program's memory.

Besides the errors xas99 would report, it refuses a `SBO`/`SBZ`/`TB` displacement outside -128 to 127, a shift count over 15, an `XOP` level over 15, an `LDCR`/`STCR` count outside 1 to 16, a `BYTE` outside -128 to 255 and a name over 12 characters. **One difference from xas99 is deliberate:** xas99 assembles `LDCR Rn,16` as `>3401` (which is `STCR Rn,0`); the TI's own encoding for 16 bits is 0 (`>3001`), and that is what the workbench produces.

How long it takes

Not measured on a real TI. In the simulator (counting the instructions executed, including every disk read through TI's own controller ROM): `examples/hello`, 67 lines, 194 bytes: **1.9 million** instructions; `examples/diag`, 630 lines, 2,260 bytes: 6.8 million; `examples/modes`, 754 lines, 2,870 bytes: 8.2 million. As a guess only, at a few hundred thousand instructions a second, a small program takes seconds and a large one tens of seconds. The menu says `assembling ...` while it works.

8. Running a program, and getting back

`R` runs the program just assembled. It puts the image into two SAMS pages (`PGUSER` and the one after it) and switches to them as the kernel switches modules: your `INIT` runs with R0 holding the workbench's page, then `TICK` and `REDRAW` as the programmer's guide describes. The assembled image stays in the workbench's memory, so `R` **runs it again without assembling**.

A program shares the kernel and its screen with the workbench: the screen is not cleared for you (a program that prints without clearing leaves the menu's text around it: part 9, tutorial 1 shows the file name still visible). Start `INIT` with `CLS` if you want a clean screen.

To come back to the workbench

```
       LI   R0,PG_WB          the workbench's page (an equate in KERNEL/EQU)
       CLR  R1                its first entry
       BL   @GOMOD
```

The example programs do this on `FCTN 9` (BACK). The menu, the file name and the last result are as you left them; the editor's text is still there.

To quit

`FCTN =` is delivered to a program as the key `KQUIT`: **a program that does not look for it cannot be quit**, because the kernel only says it is pressed. The usual lines in `TICK`:

```
       CI   R0,KQUIT*256
       JNE  NOTQ
       SETO @QUITF            tell the kernel to exit
```

(The example programs all do this.) Quitting restarts the console, as from the workbench's own menu.

What a program may not do

The workbench's own state fills the first part of the application RAM (`APPRAM`): the file name and menu state at the start, the assembler's source line and the assembled image (at `APPRAM+1024`, which is what `R` runs again), the error list, and the catalog's names. **A program that writes there overwrites the workbench**, and the assembled image `R` would run again. Keep a program's variables at `APPRAM+5760` **and above**, as the programmer's guide says for programs on this route. SAMS pages: `PGFREE` and `PGFREE+1` hold the assembler's symbol table, `PGUSER` and `PGUSER+1` are your program, and everything from `PGFREE+4` is the **page pool**, which the editor draws on; **when `R` runs a program the editor gives its pages up**, so the program may use the whole pool (even `PINIT` it itself), and the editor reloads the file the next time you press `E`. `R` refuses to run while the editor holds unsaved changes. A program that never returns to the workbench loses nothing but the session.

9. Tutorials

Tutorial 1: your first program

You need the workbench running with `workbench.dsk` (or the 360K disk) in DSK1.

1. Press **N**. The field shows `DSK1.`; type `MINE/A99` and press **ENTER**. The menu shows `File: DSK1.MINE/A99`.
2. Press **E**. The editor opens with an empty file (the status line reads `line 1 of 1`).
3. Type the program, pressing **ENTER** at the end of each line. Start each instruction line with spaces (the editor does not indent):

```
       COPY "kernel.equ"
       COPY "macros.a99"
       AORG >C000
       DATA INIT,TICK,INIT,0
INIT   .enter
       LI   R0,2*COLS+2
       LI   R1,MSG
       LI   R2,(RTEXT*16+RBG)*256
       BL   @PUTS
       .leave
       RT
TICK   RT
MSG    STRI 'Built on the TI'
```

If you make a typing mistake, `CTRL H` deletes backwards, `FCTN 3` deletes a line, and the arrows move. The screen looks like the editor picture in part 6.

4. Press **CTRL A**. It saves (`MINE/A99` now exists on the disk), returns to the menu and assembles:

```
  SAMS Kernel   workbench

  File: DSK1.MINE/A99

  N  type a file name          C  the catalog of the disk
  E  edit the file             A  assemble it         R  run the program
  FCTN =  quit


  Assembled:  50   bytes at >C000
```

5. Press **R**. The program runs; row 2 now reads `BUILT ON THE TI` (capitals, because ALPHA LOCK was down):

```
  SAMS Kernel   workbench

  BUILT ON THE TI/A99

  N  type a file name          C  the catalog of the disk
  ...
```

The tail `/A99` is the old file-name line showing through: the program printed on row 2 without clearing the screen (part 8). There is no way back to the workbench from this program, because it does not call `GOMOD` (part 8) and does not handle `KQUIT`, so it ends the session: reset the console. Tutorial 3 shows the version that behaves.

Tutorial 2: an assembly with errors

Type a source with mistakes (name it `BAD/A99`, or any name):

```
       COPY "kernel.equ"
       AORG >C000
       DATA INIT,TICK,INIT,0
INIT   LI   R0,2*COLS+2
       FROB R1,R2
       BL   @NOSUCH
       JMP  FAR
TICK   RT
INIT   RT
```

Name it (`N`) and press **A**:

```
SAMS Kernel   workbench

File: DSK1.BAD/A99

N  type a file name          C  the catalog of the disk
E  edit the file             A  assemble it         R  run the program
FCTN =  quit


6   errors:
9     label defined twice
5     unknown operation
6     undefined symbol
7     jump out of range
7     undefined symbol
9     label changed between passes
```

How to read it: the first column is the **line number in the file** (the first line of the file is line 1), the second the message. Line 5 has an unknown operation (`FROB`); line 6 uses a label that does not exist; line 9 defines `INIT` a second time. **Fix the first mistakes and assemble again**, because some errors are consequences of others: here line 7's `JMP FAR` says *both* `undefined symbol` (FAR does not exist) *and* `jump out of range` (it takes FAR to be at address 0, a long way off), and `label changed between passes` on line 9 follows from the duplicate label. Press **E**, correct, **CTRL A**. The list shows the first 18 errors; the assembler remembers 24 and counts all of them (the `6   errors:` figure).

A line number marked `(in a COPYed file)` is a line of an included file, not of yours. The marker also appears beside `file not found` for a `COPY` whose file is missing (the line is then the `COPY` line in your file).

Tutorial 3: a program that comes back

Replace the end of tutorial 1's program so that it clears the screen, goes back to the workbench on BACK, and quits on QUIT. Open `MINE/A99` (`N`, `E`) and make it:

```
       COPY "kernel.equ"
       COPY "macros.a99"
       AORG >C000
       DATA INIT,TICK,INIT,0
INIT   .enter
       LI   R1,(RTEXT*16+RBG)*256
       BL   @CLS                  a clean screen first
       LI   R0,2*COLS+2
       LI   R1,MSG
       LI   R2,(RTEXT*16+RBG)*256
       BL   @PUTS
       .leave
       RT
TICK   .enter
       CI   R0,KBACK*256          BACK (FCTN 9)?
       JEQ  GOBACK
       CI   R0,KQUIT*256          QUIT (FCTN =)?
       JNE  T9
       SETO @QUITF                tell the kernel to exit
T9     .leave
       RT
GOBACK LI   R0,PG_WB              back to the workbench
       CLR  R1
       BL   @GOMOD
       JMP  T9
MSG    STRI 'Built on the TI. BACK returns.'
```

`CTRL A` assembles it (107 bytes), `R` runs it: the screen is clean with the message; **FCTN 9** returns to the menu (the menu shows the last result, `Assembled:  107  bytes at >C000`), where `R` runs it again without reassembling. This is exactly the program the manual's author ran through the simulated workbench to write this tutorial.

Tutorial 4: change an example

Open `BEEP/A99` (`C`, pick it, ENTER; or `N`). It plays the notes C to C on keys 1 to 8. Find the table of tone values near the top and change one; assemble (`CTRL A`), run (`R`), press the key, listen. The comments in each program say what each part does, and part 10 lists what each is an example of.

Tutorial 5: a program of two files

Put a shared piece in its own file (`COPY` brings it in where it is written):

```
* in MINE/A99, after the macros line:
      COPY "mylib.a99"          reads MYLIB/A99 from the same disk
```

and `MYLIB/A99` holds, say, a routine and its constants (no `AORG`: it is placed where the `COPY` stands). A `COPY`d file can `COPY` another, three levels deep. If the include is missing the assembler says `file not found` at the `COPY` line.

On a TIPI

Map `DSK1.` to a directory on the Pi that holds the workbench's files (the TIPI's own configuration does this), or use `PI.` paths in the name (`PI.PROJ.HELLO/A99`). Includes are looked for beside the main file first (`PI.PROJ.KERNEL/EQU`), then on the bare device (`PI.KERNEL/EQU`, `DSK1.KERNEL/EQU`): one `KERNEL/EQU` can serve several directories. `skdisk.py --dir` writes the same files as TIFILES for a TIPI directory. A TIPI asks the Pi for every record, so assembling there is slower in ways that have not been measured. Tested against a model of a TIPI, not a real one.

10. The programs on the disks

Each assembles and runs on the TI (the tests do exactly that). Each is the example for one part of the programmer's guide, and its source is commented to be read.

Program	What it shows
`HELLO/A99`	the smallest program: prints a line, echoes keys, quits on QUIT (159 lines; most of it comments)
`BEEP/A99`	sound: keys 1-8 play the notes C to C on voice 1 (`SNDTONE`), 0 silences it, BACK returns
`MODE/A99`	video: 1 shows the screen in Text 80, 2 in Text 40 (`VMODE`, `TXCOLS`): the same code draws in either
`PAGING/A99`	SAMS: takes four pages from the page pool, writes each page's number into it through a window, checks that each kept its own
`COPY/A99`	files: two open at once; ENTER copies `DSK1.SRC` to `DSK1.DST` a record at a time (put a text file named `SRC` on the disk)
`ASK/A99`	`LINEIN`: asks for a name, with editing and a history of earlier entries (the up and down arrows)
`CLOCK/A99`	a TIPI's clock: ENTER reads the date and time (without a TIPI the screen says so)
`JOY/A99`	the two joysticks and a switch that makes a stick type arrow keys
`LINEIN/A99`	the `LINEIN` routine itself, an optional include: `COPY` it into a module that uses it
`PROBE1/A99`	a source that uses every instruction in every operand form: it is what the assembler is tested against
`REVERS/A99`	Reversi against the computer: a whole program (about 1,000 lines; 3,392 bytes assembled; under a minute and a half at the simulator's estimate)
`MODES/A99`	every display mode, one screen each (about 750 lines; 2,870 bytes)

The 90K disks hold the small ones; `examples.dsk` the two long ones; `workbench-360k.dsk` all of them and the workbench's own sources (`WB`, `ED`, `ASM`) to read. A program that needs equates only another application's build has (a song's page, for instance) cannot assemble here.

11. Messages and what to do about them

The assembler's errors

Each line of the list is `line number`, `message`. What each message means, with the commonest cause and what the assembler really reported for each (checked by assembling one small source per message):

Message	Means	The commonest causes, and the fix
`unknown operation`	the word where an operation should be is not one	a misspelt mnemonic (`FROB`); **an instruction at column 1**, where it is a label (`MOV R1,R2` starting in column 1 reports this, with `R1,R2` taken as the operation); `DEF` or a user macro (`.defm`), which are not supported and are reported this way
`bad operand`	an operand is not valid for the instruction	a register number over 15 (`LI R16,1`, which may also report `undefined symbol`); a missing operand (`LI R0`, which may also report `bad expression`); the wrong form for the instruction
`undefined symbol`	a name is used that is not defined	a typo in a label; **`COPY "kernel.equ"` missing**, or a `KERNEL/EQU` from another build that lacks the name; a name defined *later* in an `EQU` or

		`AORG` (these need their value already)
`label defined twice`	the same label at two places	copy-and-paste; or a local-looking name (`LOOP`) used twice: give each a different name
`jump out of range`	a `JMP`/`JEQ`/... is farther than -128 to +127 words (about 254 bytes), or goes to an odd address	use `B @LABEL` instead; for a conditional jump, jump over a `B:` `JNE SKIP` / `B @FAR` / `SKIP`. Also appears with `undefined symbol` when the target does not exist
`value out of range`	a number does not fit its field	`SBO`/`SBZ`/`TB` displacement outside -128..127; a shift count over 15; an `XOP` level over 15; an `LDCR`/`STCR` count outside 1..16; a `BYTE` outside -128..255
`file not found`	the file cannot be opened	a `COPY` of a file that is not on the disk (check the name: dots become slashes) or a typo; the line shown is the `COPY` line
`program over 4096 bytes`	the image is larger than the assembler's buffer	a `BSS` or table that is too big; split the program (a bigger program is built on a PC). A `BSS` far over the limit reports once for every byte over, so the count is large: the list still shows only 18
`bad AORG`	the `AORG` value is not usable	an `AORG` with a name not defined yet (`AORG NOSUCH`) or an expression that is not an address
`name over 12 characters`	a label or symbol is too long (reported for the line; `ABCDEFGHIJKLM RT` gives it)	shorten it
`bad expression`	an expression is malformed	an unbalanced parenthesis (`(1+`), a missing operand, a stray character
`EQU needs a label and a value`	`EQU` with no label, or a value not yet defined	put the label at column 1; define what the value uses first
`label changed between passes`	a label had one address in pass 1 and another in pass 2	usually a **consequence** of an earlier error (a duplicate label produces it): fix those first. Alone, it means something whose size or value is different in the two passes, for instance an `EQU` or `AORG` that uses a symbol defined further down

Cascades. The first error often causes the others. Fix from the top of the list and assemble again.

Counting. The heading `n errors:` counts every error; the list shows the first 18; the assembler keeps 24.

The shell's and the editor's messages

Message	Where	Means
`assembling ...`	menu	work in progress
`nothing assembled to run`	menu	`R` before a successful `A`
`unsaved changes: E, CTRL S, then run`	menu	edited since the last assembly: save and assemble first
`the program must be at >C000 and fit 8K`	menu	the image's first `AORG` is not `>C000`
`no files, or the disk could not be read`	catalog	no disk, no files, or a read error
`saved`	editor	saved
`the file could not be written`	editor	write error (full, protected, absent)
`unsaved changes: CTRL S saves, CTRL X again leaves without saving`	editor	`CTRL X` with changes
`the file is longer than 2,040 lines: ...`	editor	only part loaded, cannot be saved

When something is wrong with the machine

You see	Likely
a refusal screen naming the F18A	no F18A, or it is not responding; with a PICO9918 check the settings the checklist lists
a refusal screen about SAMS	fewer than 512K, or the card is not at the usual place

a refusal screen about a bad page	a SAMS page fails its test: the screen says which
(disk start) `THE PROGRAM CANNOT BE READ FROM THE DISK`	the bootstrap's bank files `WBENCH/B01...` are not on DSK1 to DSK3, or cannot be read
the assembly never ends	very large or looping `COPY`; the three-level limit; a disk that stops answering
a program that ran in the workbench does not work as a cartridge	built against another build's `KERNEL/EQU`, or it relies on workbench pages: rebuild on the PC with `skbuild.py`

12. Limits at a glance

editor	2,040 lines of 80 columns (163,200 bytes); a longer file loads in part and cannot be saved
file name	31 characters
catalog	the first 20 files
assembler image	4,096 bytes, one run from the first `AORG`
symbols	about 580 in all (the 341 of `KERNEL/EQU` included); names 12 characters
`COPY`	three levels
errors	the list shows 18, the assembler keeps 24
a program	**one module** of at most 4,096 bytes assembled (two SAMS pages when run), at `>C000`; no data pages, no compiled music, no own font; no second module
not supported	user macros, `DEF`, `REF`, relocatable output, listings, a cartridge image

13. Quick reference

Keys

Console keys	Name
`FCTN 1` DEL, `FCTN 2` INS, `FCTN 3` ERASE	delete character, insert/overwrite, delete line
`FCTN 4` CLEAR	(not used by the workbench)
`FCTN 5` BEGIN, `FCTN 6` PROC'D	start, end of line
`FCTN 7` AID, `FCTN 8` REDO	up, down 20 lines
`FCTN 9` BACK	leave a field or the catalog; a program's way back
`FCTN =` QUIT	leave the workbench
`FCTN E S D X`	up, left, right, down

Where	Key	
menu	`N C E A R`	name, catalog, edit, assemble, run
name field	`ENTER`, `FCTN S`/`FCTN 1`, `FCTN 9`	done, delete, leave
catalog	`FCTN E`/`FCTN X`, `ENTER`, `FCTN 9`	pick, choose, return
editor	`CTRL S`, `CTRL A`, `CTRL X`	save, save and assemble, menu (twice if unsaved)
editor	`CTRL H`, `FCTN 1`, `FCTN 3`, `ENTER`	backspace, delete character, delete line, split line

The smallest program

```
       COPY "kernel.equ"
       COPY "macros.a99"
       AORG >C000
       DATA INIT,TICK,INIT,0
INIT   .enter
       ...
       .leave
       RT
TICK   RT
```

`INIT` runs once, `TICK` once a frame (60 times a second) with the key in R0, `REDRAW` when the screen must be drawn again.

The workflow

`N` name, `ENTER`; `E` edit; `CTRL A` save and assemble; if errors, `E`, fix the first, `CTRL A`; `R` run; `FCTN 9` back; `E` again.

14. The PC route, and where to read next

Every step has a PC counterpart, and the source is the same on both:

[TI] in the workbench	[PC]
`N`, a file name; `E` to edit	any editor
`A` assembles	`python3 tools/skbuild.py APPDIR` (xas99)
`R` runs	`python3 tools/sim9900.py build/APP/APP8.bin --keys ...`, js99er, Classic99 or a real TI
the catalog	your file manager; `xdm99.py disk.dsk` lists a disk image
`COPY "kernel.equ"` reads `KERNEL/EQU`	the build writes `kernel.equ`

To build the workbench's disks yourself: `python3 tools/skbuild.py examples/workbench` makes `build/workbench/workbench8.bin`; `python3 tools/skdisk.py work.dsk --kernel-equ build/workbench/kernel.equ FILES...` makes a disk holding `KERNEL/EQU` and your sources (add `--geometry dsdd` for 360K); `tools/skbuild.py ... --disk --disk-files FILES...` makes the disk-started workbench.

Read next: PROGRAMMERS-GUIDE.md (writing the programs: part 1 is Hello, kernel); WORKBENCH.md (how the editor and assembler are built and what they cost); DISKSTART.md (starting a program from disk); DEVELOPING-ON-A-TI.md (the first-run ladder, refusal screens, debugging on a real TI); HARDWARE-CHECKLIST.md (what has not been tried on a real machine, and how to try it); RATIONALE.md (why the kernel is the way it is).